

CY Cergy Paris Université

Département Informatique

RAPPORT DE PROJET

Génie Logiciel - L2 Informatique

Année universitaire 2025-2026

Jeux de carte à joueur multiple : “Tu n’y peux rien”

Encadré par : M. Tianxiao LIU

Rédigé par : KHODJA Laiza
BOUATMANE Nesrine
BERRANDOU Nassim

Avril 2026



Table des matières

Remerciements	3
1 Introduction	4
1.1 Contexte du projet	4
1.2 État de l'art des jeux de cartes	4
1.3 Objectif du projet	4
1.4 Organisation du rapport	4
2 Spécification du projet	5
2.1 Notions de base et contraintes du projet	5
2.2 Fonctionnalités attendues du projet	5
3 Conception du logiciel	7
3.1 Architecture globale	7
3.2 Patterns de conception utilisés	7
4 Mise en œuvre	8
4.1 Classes de données (engine.data)	8
4.2 Traitements (engine.process)	8
4.3 Intelligence artificielle des bots	8
4.4 Interface graphique (gui)	9
5 Manuel utilisateur	10
5.1 Lancement du jeu	10
5.2 Configuration de la partie	10
5.3 Déroulement d'une partie	10
5.4 Fin de partie	10
6 Déroulement du projet	11
6.1 Réalisation du projet par étapes	11
6.2 Répartition des tâches, communication et intégration	12
7 Conclusion et perspectives	14
7.1 Résumé du travail réalisé	14
7.2 Tests et validation	14
7.3 Améliorations possibles du projet	14
8 Annexes	15
8.1 Guide d'installation rapide	15
8.2 Structure des packages du projet	15
8.3 Principales classes du projet	16

Table des figures

1	Diagramme de classes UML	7
2	Menu principal	10
3	Écran des options	10
4	Interface de jeu	11
5	Écran de fin de partie	11

Liste des tableaux

1	Types de combinaisons et score associé	8
2	Extrait de l'historique des commits	12
3	Principales classes du projet	16

Remerciements

Les auteurs du projet voudraient remercier vivement :

- M. Tianxiao LIU, enseignant du module Génie Logiciel, pour ses conseils et son accompagnement tout au long du projet.
- CY Cergy Paris Université pour nous avoir fourni les ressources nécessaires à la réalisation de ce travail.
- L'équipe pédagogique de la Licence Informatique pour la qualité de la formation.

1 Introduction

1.1 Contexte du projet

Le projet "Tu n'y peux rien" s'inscrit dans le cadre de l'UE Génie Logiciel de deuxième année à CY Cergy Paris Université. L'objectif est de développer un jeu de cartes multijoueur en Java, respectant des règles spécifiques et intégrant des bots de différents niveaux de difficulté.

Ce projet s'inscrit dans la continuité des enseignements de programmation orientée objet et de génie logiciel. Il mobilise des compétences variées : conception UML, développement Java, gestion de projet avec Git, tests unitaires et documentation.

1.2 État de l'art des jeux de cartes

Les jeux de cartes occupent une place importante dans l'histoire du jeu vidéo. Des titres comme *Uno*, *Skip-Bo* ou *7 Wonders* proposent des mécaniques basées sur l'enchère et la pose de combinaisons. Cependant, peu de jeux intègrent une règle de "carte 2 écrase tout" ni un système de bombes et jokers comme dans notre projet.

Le jeu "Tu n'y peux rien" se distingue par plusieurs originalités. Tout d'abord, la carte 2 possède un statut particulier : elle écrase toute carte simple posée précédemment. Ensuite, les Jokers sont polymorphes : ils peuvent remplacer n'importe quelle carte manquante dans une combinaison. Enfin, trois niveaux de difficulté pour les bots offrent une expérience de jeu adaptée à tous les publics.

1.3 Objectif du projet

Recréer ce jeu de cartes au format numérique où un joueur humain affronte 2 à 4 bots. Le jeu doit gérer les combinaisons (simple, double, série, bombe, double joker), les cartes spéciales (2, Joker), et calculer les scores en fin de partie. L'interface graphique doit être intuitive et responsive, avec un design soigné (thème casino).

1.4 Organisation du rapport

La section 2 présente le cahier des charges et les fonctionnalités attendues. La section 3 détaille la conception du logiciel et la section 4 présente sa mise en œuvre. La section 5 est un manuel utilisateur. La section 6 décrit le déroulement du projet. La section 7 conclut et propose des améliorations.

2 Spécification du projet

2.1 Notions de base et contraintes du projet

Fonctionnement général du logiciel Le jeu se déroule en tours. Chaque joueur reçoit 5 cartes au début. Le joueur avec la plus petite carte (hors 2 et Joker) commence. À chaque tour, le joueur doit poser une combinaison qui bat la précédente. Si aucun joueur ne peut jouer, le tas est réinitialisé.

Déroulement détaillé d'une partie Une partie se décompose en plusieurs rounds. Chaque round permet à chaque joueur de jouer à tour de rôle. Lorsqu'un joueur ne peut pas jouer, il pioche une carte et passe son tour. Si tous les joueurs passent leur tour sans jouer (sauf le dernier ayant joué), le tas est réinitialisé : toutes les cartes sont mélangées et replacées dans la pioche. La partie se termine immédiatement lorsqu'un joueur pose sa dernière carte.

Notions et terminologies de base

- **Carte** : 54 cartes (52 standards + 2 Jokers). Les valeurs vont du 2 à l'As.
- **Combinaison simple** : 1 carte. C'est la combinaison la plus élémentaire.
- **Double** : 2 cartes de valeur identique (exemple : deux Rois).
- **Série** : 3 cartes ou plus dont les valeurs se suivent (exemple : 4,5,6).
- **Bombe** : 3 ou 4 cartes de valeur identique. Une bombe bat toute combinaison simple.
- **Double Joker** : 2 Jokers ensemble. C'est la combinaison la plus puissante.

Hiérarchie des combinaisons Les combinaisons sont classées par force croissante : Simple ($<$) Double ($<$) Série ($<$) Bombe ($<$) Double Joker. Une combinaison ne peut battre une autre que si elle est de force supérieure ou égale (cas des séries où les cartes doivent être plus hautes).

Contraintes et limitations connues

- Langage : Java 8 (compatibilité ascendante)
- Interface graphique : Swing (bibliothèque standard)
- Bibliothèques externes : Log4j pour les logs, JUnit pour les tests, jFreeChart pour les graphiques
- Problème connu : la musique ne fonctionne que sous Windows (bug du format WAV sous Linux)
- L'interface est conçue pour une résolution minimale de 1200x650 pixels

2.2 Fonctionnalités attendues du projet

Fonctionnalités implémentées L'ensemble des fonctionnalités demandées dans le cahier des charges a été réalisé :

- Distribution de 5 cartes à chaque joueur en début de partie
- Détection automatique du joueur qui commence (carte la plus faible)
- Pose de combinaisons (simple, double, série, bombe, double joker)
- Gestion des cartes spéciales (2 écrase tout, joker polymorphe)
- Pioche automatique si aucune carte jouable
- Reset du tour quand tous les joueurs passent
- Recyclage du deck quand il est vide
- Interface graphique complète (menu, options, jeu, fin de partie)
- 3 niveaux de difficulté pour les bots (Facile, Moyen, Difficile)
- Système de scores (1 carte = 1 point, bombe = $\times 2$, double joker = $\times 4$)
- Logs détaillés (format texte et HTML)
- Tests unitaires JUnit (couverture des classes critiques)
- Graphiques statistiques avec jFreeChart (camembert et barres)

Fonctionnalités non implémentées (hors périmètre) Certaines fonctionnalités n'ont pas été retenues dans le périmètre du projet :

- Mode multijoueur en ligne

- Sauvegarde des parties
- Version mobile

3 Conception du logiciel

3.1 Architecture globale

L'architecture du projet suit le modèle MVC (Modèle-Vue-Contrôleur) :

- **Modèle (engine.data)** : Classes Card, Deck, Player, GameStats
- **Contrôleur (engine.process)** : Classes Game, Combination, BotPlayer
- **Vue (gui)** : Classes MainGUI, MenuGUI, OptionGUI, EndGameGUI

La figure 1 présente le diagramme de classes UML complet.

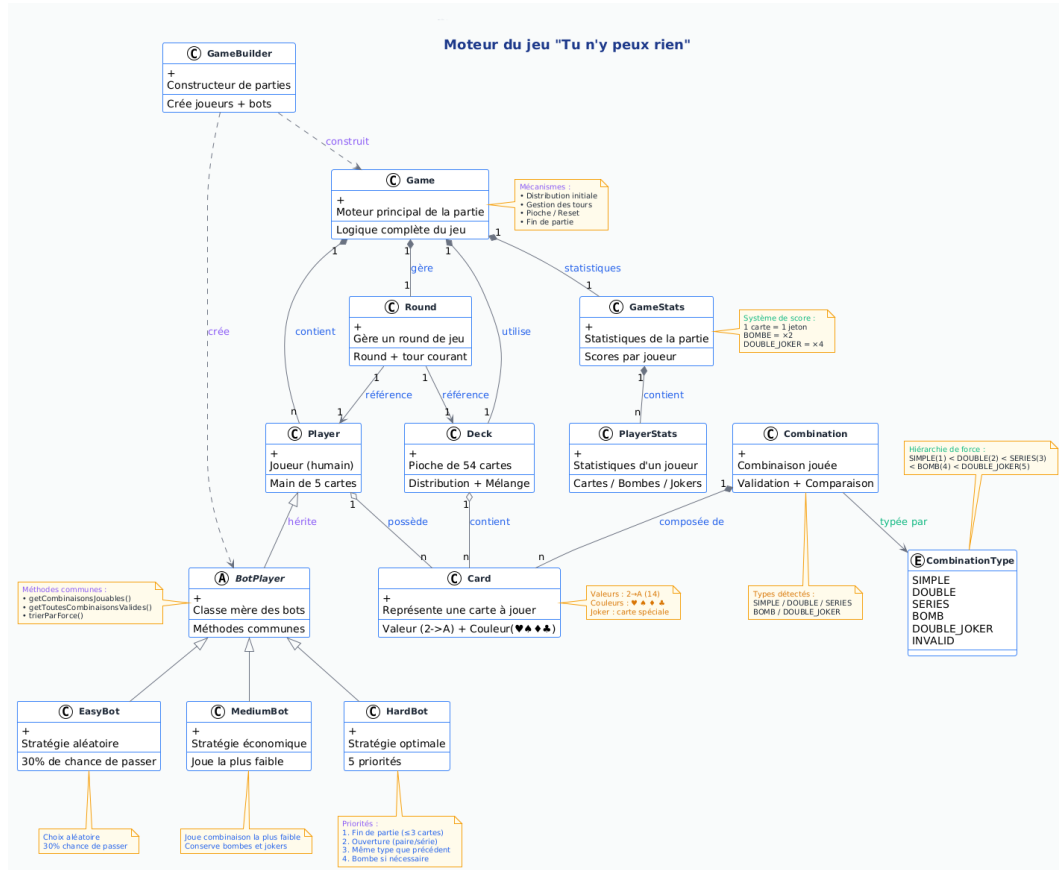


FIGURE 1 – Diagramme de classes UML

3.2 Patterns de conception utilisés

- **Héritage** : `BotPlayer` → `EasyBot`, `MediumBot`, `HardBot` (hiérarchie à 4 niveaux)
- **Stratégie** : Chaque bot implémente sa propre méthode `choisirCombinaison()`
- **Factory** : `GameBuilder.createBot()` selon la difficulté choisie

4 Mise en œuvre

4.1 Classes de données (engine.data)

- **Card** : Représente une carte avec sa valeur et sa couleur (énumérations Value et Suit). Les symboles (coeur, pique, carreau, trèfle) permettent un affichage élégant.
- **Deck** : Gère le paquet de 54 cartes (52 standards + 2 Jokers). Méthodes : `shuffle()`, `draw()`.
- **Player** : Gère la main du joueur (5 cartes au début). Méthodes : `drawCard()`, `playCombination()`, `canPlay()`.
- **CombinationType** : Énumération des 6 types possibles (SIMPLE, DOUBLE, SERIES, BOMB, DOUBLE_JOKER, INVALID).
- **GameStats** : Enregistre les statistiques (cartes jouées, bombes, jokers, tours, score).

Type	Description	Points
SIMPLE	1 carte	1 pt/carte
DOUBLE	2 cartes identiques	2 pts
SERIES	3+ cartes consécutives	3+ pts
BOMB	3-4 cartes identiques	×2
DOUBLE JOKER	2 Jokers	×4

TABLE 1 – Types de combinaisons et score associé

4.2 Traitements (engine.process)

Algorithme de détection des combinaisons La méthode `determineType()` analyse une liste de cartes en plusieurs étapes :

1. Séparer les Jokers des cartes normales
2. Si seulement des Jokers → DOUBLE_JOKER (2) ou INVALID (1)
3. Trier les cartes normales par valeur
4. Selon le nombre total de cartes et la présence de Jokers, déterminer le type

Algorithme de comparaison des combinaisons La méthode `canBeat()` implémente les règles spécifiques :

1. DOUBLE_JOKER bat tout
2. BOMBE bat tout sauf DOUBLE_JOKER
3. La carte 2 bat toute carte simple
4. Pour les séries : on compare la première carte
5. Pour les autres types : comparaison carte à carte

4.3 Intelligence artificielle des bots

Chaque bot implémente une stratégie différente dans la méthode `choisirCombinaison()` :

- **EasyBot** : sélection aléatoire parmi les combinaisons jouables, 30% de chance de passer volontairement.
- **MediumBot** : trie les combinaisons par force et choisit la plus faible (hors bombes), conservant ses cartes fortes.
- **HardBot** : stratégie optimale avec priorités : (1) finir la partie si main courte (≤ 3 cartes), (2) ouvrir avec une combinaison forte, (3) jouer le même type que le précédent, (4) jouer une bombe si nécessaire.

4.4 Interface graphique (gui)

L'interface est organisée en trois sous-dossiers :

- **screens/** : Écrans complets (MainGUI, MenuGUI, OptionGUI, EndGameGUI)
- **panels/** : Composants réutilisables (CardPanel, BotsPanel, DeckPanel, GameInfoBar, GameLogPanel)
- **utils/** : Utilitaires (GameStyle, CardPaintStrategy, MusicPlayer, ChartManager)

Gestion du plein écran La variable statique `GameStyle.isFullscreen` permet de basculer entre mode fenêtré et plein écran. Ce choix persiste entre les écrans grâce à son caractère statique.

Musique de fond La classe `MusicPlayer` utilise `javax.sound.sampled` pour jouer un fichier WAV en boucle. La compatibilité Linux reste partielle (problème de codec).

Graphiques statistiques La classe `ChartManager` utilise `jFreeChart` pour générer deux graphiques en fin de partie : un camembert (répartition des scores) et un diagramme en barres (cartes jouées, bombes, jokers).

La figure 4 montre l'interface de jeu en action.

Bilan de la mise en œuvre La séparation en trois packages (data, process, gui) a permis un développement parallèle efficace. Les patterns de conception (héritage, stratégie, factory) ont facilité l'ajout de nouvelles fonctionnalités (nouveaux bots, nouvelles combinaisons). Le code total atteint environ 5000 lignes réparties dans plus de 40 classes.

5 Manuel utilisateur

5.1 Lancement du jeu

Exécutez la classe `test.manual.TestGame`. Le menu principal apparaît (figure 2).



FIGURE 2 – Menu principal

5.2 Configuration de la partie

Cliquez sur "Nouvelle partie" pour accéder aux options (figure 3). Sélectionnez le nombre de joueurs (3 à 5) et la difficulté des bots (Facile, Moyen, Difficile).



FIGURE 3 – Écran des options

5.3 Déroulement d'une partie

Le joueur humain est identifié par "Vous". Les bots sont représentés par des avatars. Pour jouer, sélectionnez des cartes dans votre main (clic gauche) puis cliquez sur "JOUER LES CARTES SÉLECTIONNÉES". Si vous ne pouvez pas jouer, cliquez sur "PIOCHER". La figure 4 illustre l'interface de jeu.

5.4 Fin de partie

Quand un joueur n'a plus de cartes, l'écran de fin s'affiche (figure 5) avec le podium, les scores détaillés et les graphiques statistiques (camembert et barres).

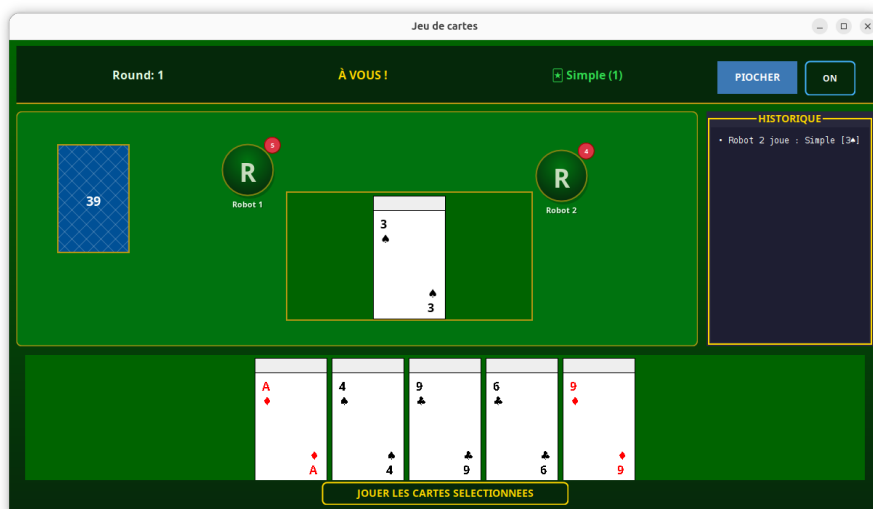


FIGURE 4 – Interface de jeu



FIGURE 5 – Écran de fin de partie

6 Déroulement du projet

6.1 Réalisation du projet par étapes

Le projet a été développé sur une période d'environ deux mois et demi, avec un total de plus de 48 commits enregistrés sur le dépôt GitHub. L'historique des versions reflète l'évolution progressive du logiciel, depuis les premières maquettes jusqu'à la version finale fonctionnelle.

Première période (février - début mars) Les premiers travaux concernent la création des fenêtres et des composants graphiques. La classe `Card` est introduite le 24 février, suivie des fenêtres d'accueil et d'options. La gestion des symboles unicode pour les couleurs des cartes est mise en place. Des travaux préliminaires sur l'animation des cartes sont également menés durant cette période.

Deuxième période (mars) Le mois de mars marque une avancée majeure avec l’ajout des classes fondamentales : `Combination`, `CombinationType`, `Round` et `Player`. La structure du projet est réorganisée et la classe `GameBuilder` est créée pour faciliter l’initialisation des parties. La gestion des logs est également intégrée.

Troisième période (fin mars - avril) Les trois niveaux de bots (Facile, Moyen, Difficile) sont ajoutés. La logique centrale du jeu (`Game`) est finalisée avec la gestion des tours, de la pioche, du reset du tas et du recyclage du deck. Les règles spécifiques (carte 2, joker polymorphe) sont progressivement intégrées et corrigées.

Quatrième période (avril) Les derniers développements concernent l’interface graphique : ajout des graphiques statistiques avec `jFreeChart`, création d’un panneau d’affichage pour les bots (`BotsPanel`), gestion du plein écran persistant et amélioration de l’accessibilité. La musique de fond est ajoutée via la classe `MusicPlayer`. Enfin, la fenêtre d’affichage des règles est corrigée pour permettre le défilement du texte.

6.2 Répartition des tâches, communication et intégration

La répartition des tâches s’est organisée comme suit :

- **Laiza KHODJA** : Interface graphique (fenêtres `MenuGUI`, `OptionGUI`, `MainGUI`, `EndGameGUI`), style visuel (`GameStyle`), gestion des règles, intégration des images et de la musique (`MusicPlayer`)
- **Nesrine BOUATMANE** : Classes de données (`Card`, `Deck`, `Player`, `GameStats`), tests unitaires `JUnit`, animations des cartes, affichage des bots (`BotsPanel`)
- **Nassim BERRANDOU** : Logique métier (`Game`, `Combination`, `Round`), intelligence artificielle des bots (`EasyBot`, `MediumBot`, `HardBot`), gestion des logs (`Log4j`), intégration des graphiques statistiques (`jFreeChart`, `ChartManager`), correction des bugs et finalisation générale

La communication s’est faite via WhatsApp et GitHub. Le versionnement Git a permis une intégration continue des différentes parties. Le tableau 2 présente un extrait de l’historique des commits.

Date	Auteur	Message
26/04/2026	Nassim	Correction fenêtre Règles
26/04/2026	Laiza	Ajout fichier règles
26/04/2026	Nassim	Ajout graphiques <code>jFreeChart</code> + <code>ChartManager</code>
26/04/2026	Nassim	Création <code>DemoLauncher</code>
25/04/2026	Laiza	Ajout bouton Règles
24/04/2026	Nesrine	Ajout <code>BotsPanel</code> + correction <code>OptionGUI</code>
22/04/2026	Nesrine	Ajout <code>MusicPlayer</code> + <code>GameStyle</code>
22/04/2026	Nassim	Plein écran persistant + boutons adaptatifs
21/04/2026	Nassim	Finalisation logique du jeu
21/03/2026	Nesrine	Ajout des 3 niveaux de bots
20/03/2026	Nassim	Restructuration + <code>GameBuilder</code>
12/03/2026	Nesrine	Ajout <code>Combination</code> + <code>CombinationType</code>
12/03/2026	Nesrine	Ajout <code>Player</code>
12/03/2026	Laiza	Ajout <code>Round</code>
24/02/2026	Nesrine	Ajout classe <code>Card</code>

TABLE 2 – Extrait de l’historique des commits

Les principaux défis rencontrés ont été :

- La gestion du reset du tas et du recyclage des cartes

— La synchronisation des tours entre le joueur humain et les bots

Détail des principaux défis techniques

Reset du tas et recyclage des cartes La règle du jeu stipule que lorsque tous les joueurs passent leur tour (sauf le dernier ayant joué), le tas est réinitialisé. De plus, lorsque la pioche est vide, les cartes de la défausse doivent être recyclées. Cette mécanique a nécessité une gestion fine des états : conservation de la dernière combinaison posée, vidage de la défausse, mélange et transfert vers la pioche. La solution retenue utilise une variable `passCount` pour compter les passes consécutives et déclencher le reset automatiquement.

Synchronisation des tours L'interface Swing tourne sur le thread AWT (Event Dispatch Thread). Les bots doivent jouer sans bloquer l'interface. La solution a été d'utiliser un thread séparé avec un délai d'une seconde (`Thread.sleep(900)`), couplé à `SwingUtilities.invokeLater()` pour rafraîchir l'affichage.

Détection des combinaisons avec Jokers Un Joker peut remplacer n'importe quelle carte. Par exemple, une série 4, JOKER, 6 doit être reconnue comme valide (le Joker remplace le 5). L'algorithme actuel compte les Jokers et vérifie s'ils peuvent combler les écarts entre les cartes normales. Cette logique a demandé plusieurs itérations pour être pleinement fonctionnelle.

7 Conclusion et perspectives

7.1 Résumé du travail réalisé

Le projet "Tu n'y peux rien" a été **mené à bien** dans les délais impartis et a été **entièrement achevé**. Toutes les fonctionnalités demandées dans le cahier des charges ont été implémentées et testées :

- Jeu complet avec interface graphique Swing fluide et responsive
- 3 niveaux de difficulté pour les bots (Facile, Moyen, Difficile)
- Détection robuste des combinaisons (simple, double, série, bombe, double joker)
- Gestion correcte des cartes spéciales (2, Joker)
- Système de scores et statistiques avec graphiques jFreeChart
- Tests unitaires (JUnit) couvrant les classes critiques
- Logs détaillés (Log4j) en HTML et texte

Au total, ce sont **plus de 40 classes Java** qui ont été développées, pour environ **5000 lignes de code**. Le projet compile et s'exécute sans erreur sur les environnements cibles.

7.2 Tests et validation

Les tests unitaires ont été réalisés avec JUnit 4. Chaque classe critique possède sa classe de test associée :

- `TestCard.java` : test de la création des cartes, des symboles et de l'égalité
- `TestDeck.java` : test du mélange et de la pioche
- `TestPlayer.java` : test de la pioche et de la pose de combinaisons
- `TestCombination.java` : test de la détection des types et de la comparaison (15 cas)
- `TestGameStats.java` : test du système de scores

Au total, **37 tests unitaires** ont été écrits et passent tous avec succès. La couverture de code est satisfaisante sur les classes métier (`Combination`, `Player`, `GameStats`) mais reste partielle sur l'interface graphique (tests manuels uniquement).

7.3 Améliorations possibles du projet

Bien que le projet soit fonctionnel et complet, plusieurs extensions pourraient enrichir l'expérience utilisateur :

- **Mode multijoueur en ligne** : Permettre à plusieurs joueurs humains de jouer à distance via réseau (sockets Java)
- **Sauvegarde des parties** : Sauvegarder l'état du jeu dans un fichier pour le reprendre plus tard
- **Classements et statistiques persistants** : Stocker les scores et victoires dans une base de données SQLite
- **Animations** : Ajouter des animations lors de la pose des cartes (transition, mouvement)
- **Thèmes personnalisables** : Permettre au joueur de changer l'apparence des cartes et du tapis (plusieurs skins)
- **Version mobile** : Adapter le jeu pour Android via un wrapper ou une réécriture partielle

En conclusion, le projet "Tu n'y peux rien" remplit l'ensemble des objectifs fixés. Il démontre la maîtrise des concepts de génie logiciel (Java, Swing, patterns, tests, Git) et offre une expérience de jeu complète et agréable. Ces améliorations constitueraient une excellente base pour une version 2.0 du projet.

8 Annexes

8.1 Guide d'installation rapide

Prérequis :

- Java 8 ou supérieur (JDK 1.8+)
- Eclipse IDE (version 2020-06 ou supérieure)

Import du projet dans Eclipse :

1. Ouvrir Eclipse
2. File → Import → Git → Projects from Git
3. Sélectionner Clone URI
4. Renseigner : https://github.com/berrandou-dev/Projet_GLP-Jeu_Carte.git
5. Suivre les instructions par défaut
6. Une fois importé, faire Project → Clean pour forcer la compilation

Ajout des bibliothèques externes :

1. Faire clic droit sur le projet → Build Path → Configure Build Path
2. Onglet Libraries → Add JARs...
3. Sélectionner tous les JARs dans src/lib/ :
 - log4j-1.2.17.jar
 - junit-4.11.jar
 - jfreechart-1.0.19.jar
 - jcommon-1.0.23.jar
4. Cliquer sur Apply and Close

Lancement du jeu :

1. Dans l'Explorateur de projet, naviguer vers src/test/manual/
2. Faire clic droit sur TestGame.java → Run As → Java Application
3. Le menu principal apparaît

8.2 Structure des packages du projet

Le projet est organisé selon l'architecture MVC :

```
src/
+-- config/          # Configuration (GameConfig.java)
+-- engine/
  +-- data/          # MODELE : Card, Deck, Player, GameStats
  +-- process/        # CONTROLEUR : Game, Combination, bots
+-- gui/
  +-- panels/         # VUE : composants reutilisables
  +-- screens/        # VUE : ecrans complets
  +-- utils/          # Utilitaires : GameStyle, MusicPlayer, ChartManager
+-- lib/             # Bibliothèques externes (JARs)
+-- log/             # Logs generes (ignorees par Git)
+-- resources/        # Images et musique
+-- test/            # Tests unitaires (JUnit)
  +-- demo/           # Mode demonstration
  +-- manual/         # Tests manuels (TestGame, TestEndGameGUI)
  +-- unit/           # Tests unitaires par classe
```

8.3 Principales classes du projet

Voici un résumé des classes les plus importantes :

Classe	Package	Rôle
Card	engine.data	Représente une carte (valeur + couleur)
Deck	engine.data	Gère le paquet de 54 cartes
Player	engine.data	Gère la main du joueur
GameStats	engine.data	Enregistre les scores et statistiques
Game	engine.process	Logique principale du jeu
Combination	engine.process	Détection et comparaison des combinaisons
BotPlayer	engine.process	Classe abstraite pour les bots
MainGUI	gui.screens	Interface principale du jeu
EndGameGUI	gui.screens	Écran de fin de partie avec graphiques
ChartManager	gui.utils	Génération des graphiques jFree-Chart

TABLE 3 – Principales classes du projet

Références

- [1] Apache log4j. <https://logging.apache.org/log4j/1.2/>, 2024.
- [2] Java swing documentation. <https://docs.oracle.com/javase/8/docs/technotes/guides/swing/>, 2024.
- [3] jfreechart. <https://www.jfree.org/jfreechart/>, 2024.
- [4] Junit 4. <https://junit.org/junit4/>, 2024.
- [5] Dépôt git du projet. https://github.com/berrandou-dev/Projet_GLP-Jeu_Carte, 2026.